

Ex. No.: 9 DEADLOCK AVOIDANCE

Aim:

To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i
2. Find an i such that both: finish[i]=false and Needi<= work
3. If no such i exists go to step 6
4. Compute work=work+allocationi
5. Assign finish[i] to true and go to step 2
6. If finish[i]==true for all i, then print safe sequence
7. Else print there is no safe sequence.

Program

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Completed

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main() {
3     int n, m, i, j, k;
4
5     printf("Enter number of processes: \n");
6     scanf("%d", &n);
7
8     printf("Enter number of resource types: \n");
9     scanf("%d", &m);
10
11     int alloc[n][m], max[n][m], need[n][m];
12     int avail[m], finish[n], safeSeq[n];
13
14     // Allocation Matrix
15     printf("Enter Allocation Matrix:\n");
16     for(i = 0; i < n; i++) {
17         for(j = 0; j < m; j++) {
18             scanf("%d", &alloc[i][j]);
19         }
20     }
21
22     // Max Matrix
23     printf("Enter Max Matrix:\n");
24     for(i = 0; i < n; i++) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice
Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language: C (GCC 9.2.0)

```

69         }
70     }
71     safeSeq[count++] = i;
72     finish[i] = 1;
73     found = 1;
74 }
75 }
76 }
77
78 // If no process could be executed
79 if(found == 0) {
80     printf("System is NOT in a safe state (Deadlock may occur).");
81     return 0;
82 }
83
84 // Safe state
85 printf("System is in a SAFE STATE.\nSafe Sequence: ");
86 for(i = 0; i < n; i++) {
87     printf("%d ", safeSeq[i]);
88 }
89
90 return 0;
91 }
92

```

Test Cases 2
Run Sample Cases
Run All Cases

Sample Test Case

Test Case - 1

Input :

3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0

Expected Output :

Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).

Result:

Thus the program to find out a safe sequence using Banker's algorithm has been successfully executed.